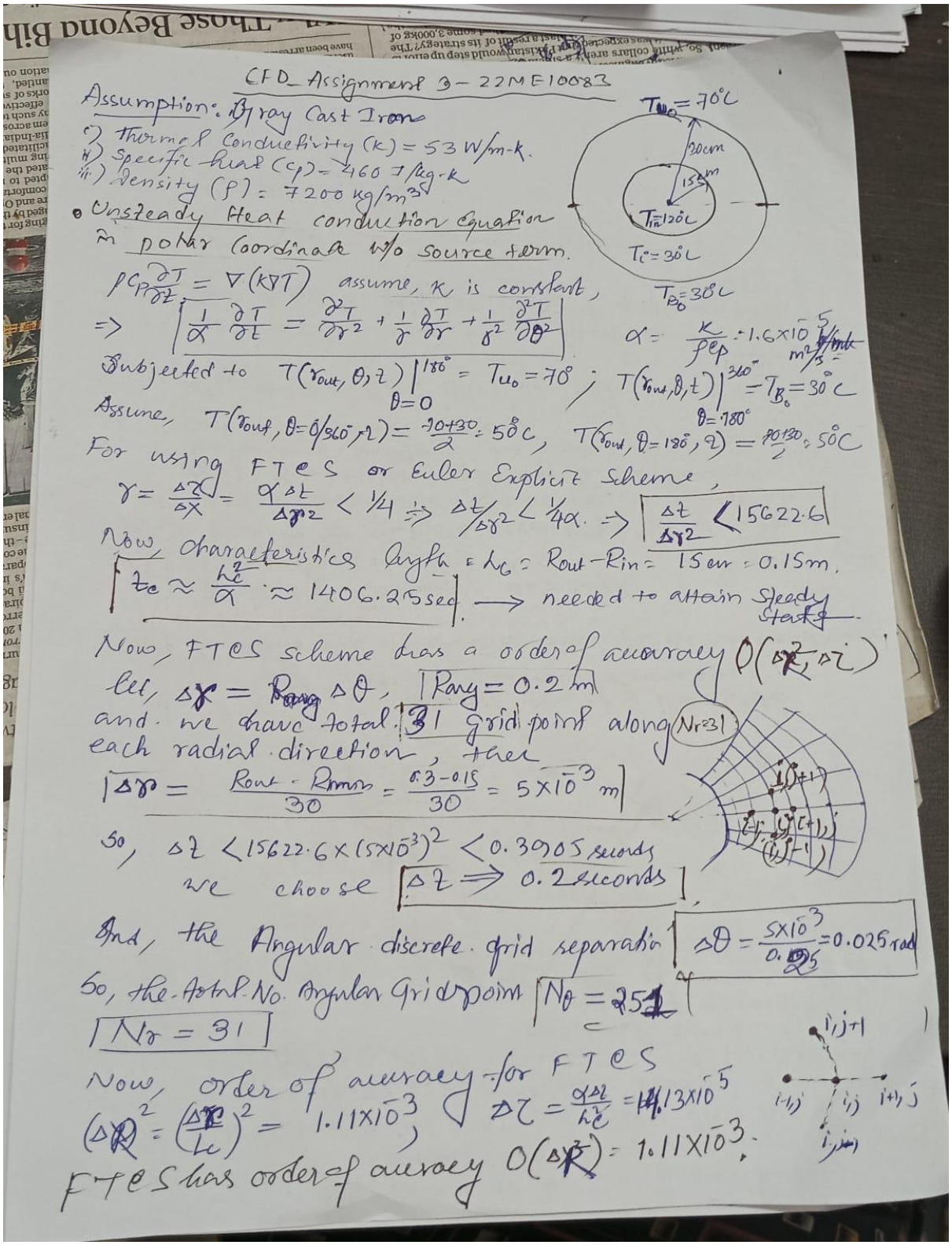
CFD Assignment 3 on Unsteady Thermal Diffusion on Circular Disk

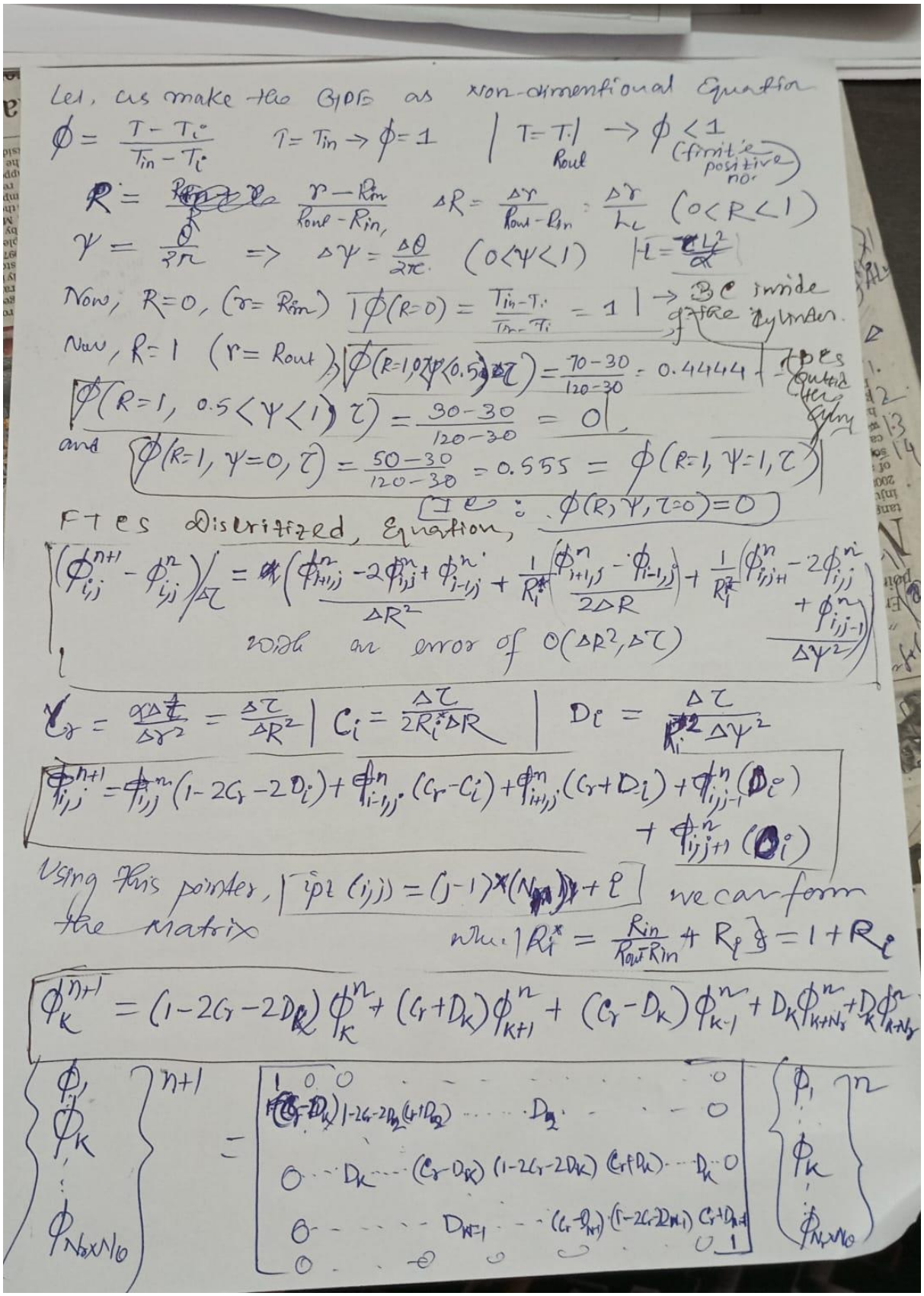
- By Souvik Sarkar (22ME10083)

Derivation and Accuracy Analysis:

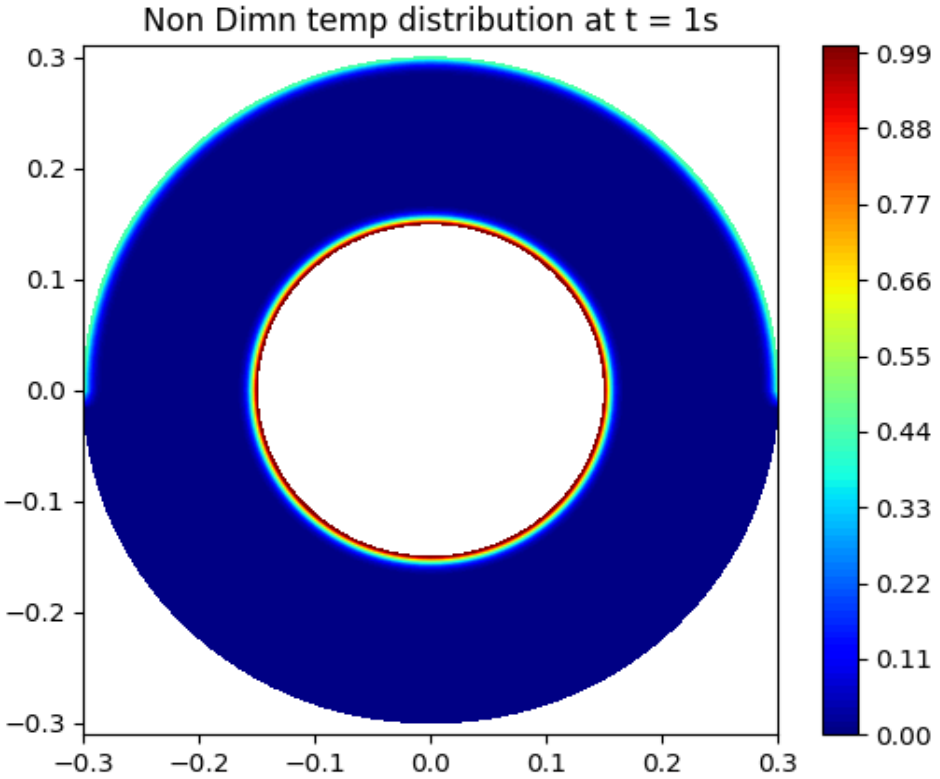


Formulation & Discretization using FTCS or Euler Explicit Scheme:

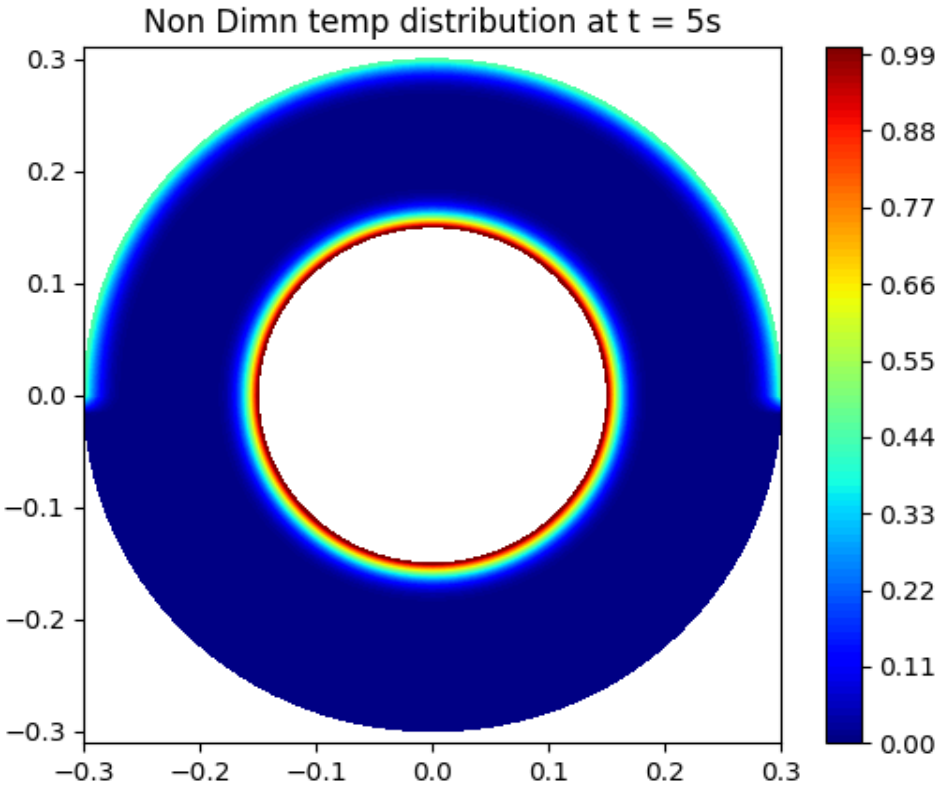
First it is non dimentionalized and make it vector using pointer



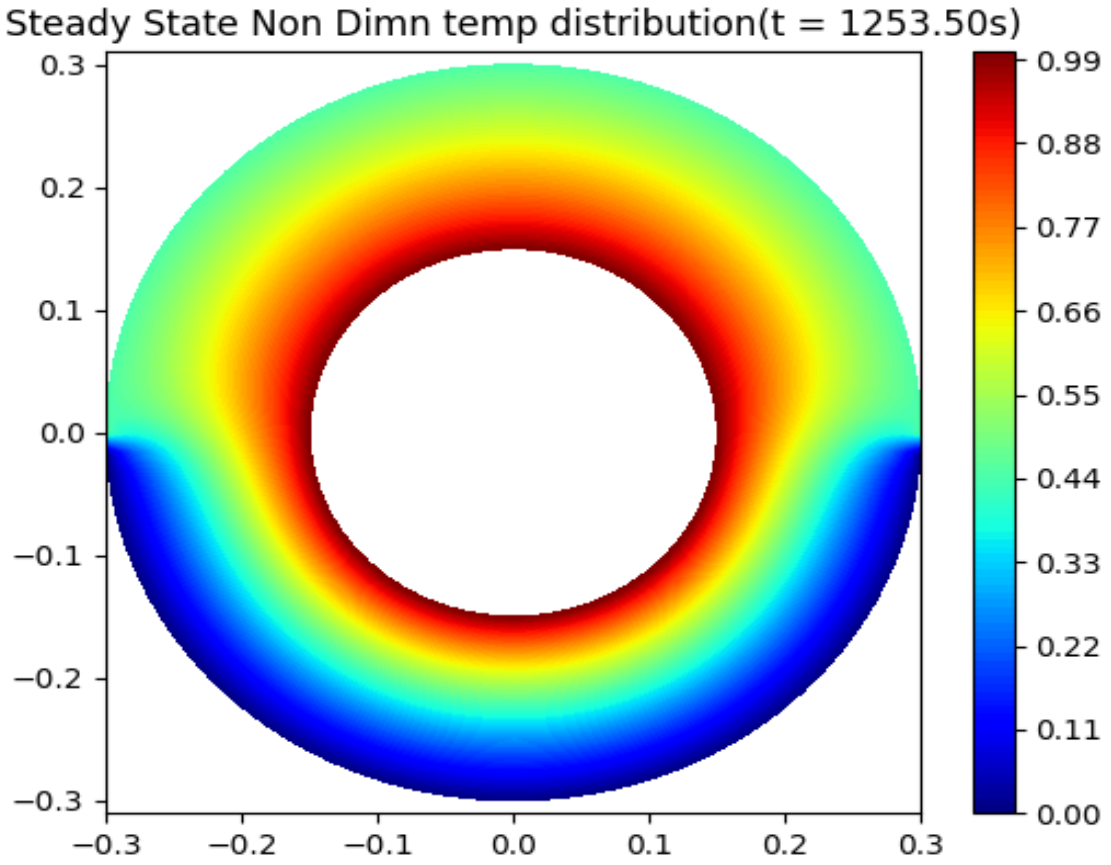
Results and Discussion : 1. Snap Shot at 1s :



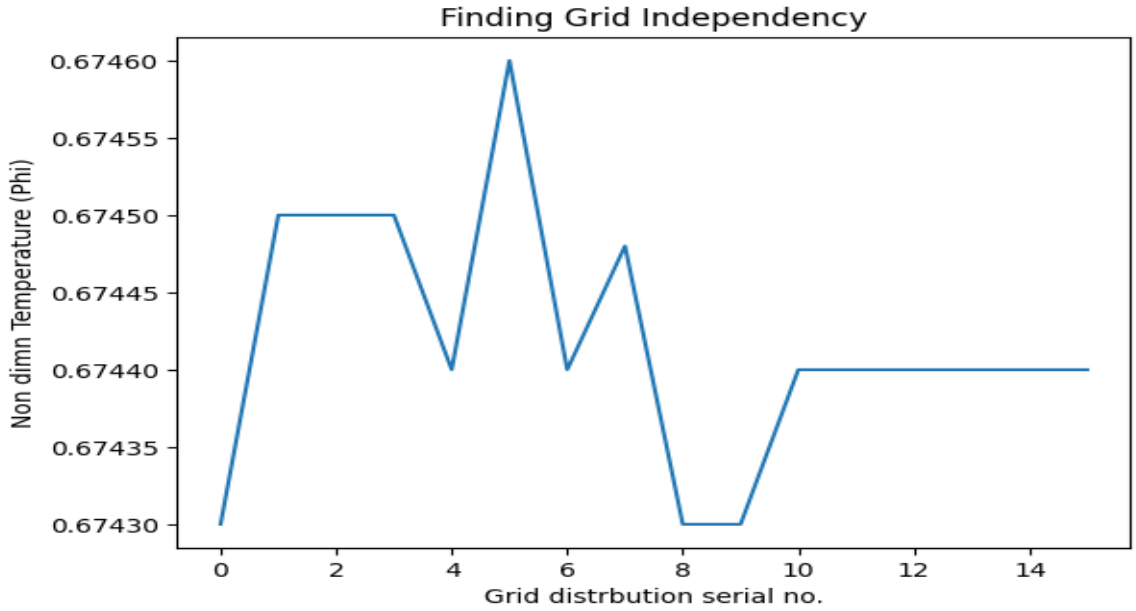
2. Snapshot at 5 seconds :



3. Steady state plot :



4. Grid Independency :



** **Grid Independency attain at '10' -> Nr = 41 and Ntheta = 121 resolution**

5. Working Code Snippet :

```
# ------------------------------------------------------------------------
#                       CFD Assignment 3 by 22ME10083
#-------------------------------------------------------------------------
## NON DIMENTIONAL APPROACH

import numpy as np
import matplotlib.pyplot as plt

# 1. Material Properties for Cast Iron
k = 53.0            # Thermal conductivity (W/m.K)
rho = 7200.0        # Density (kg/m^3)
cp = 460.0          # Specific heat (J/kg.K)
alpha = k / (rho * cp) # Thermal Diffusivity ( m2/s)

# Geometric property

ri, ro = 0.15, 0.30
Lc = ro - ri
ravg = 0.25

## Analysis of stabilty condition
# assume :
Nr = 41;
dr = (ro - ri) / (Nr - 1)
print(f"dr = {dr}")

#dt = 0.2 * (1/alpha) * (1 / (1/dr**2 + 1/(ri*dtheta)**2))
dt_required = 0.25 * (1/alpha) * dr**2;
print(f"dt_required < {dt_required} so we choose 0.1s")
dt = 0.1;

# As create a huge no. of iteration so excluded
#dtheta = dr/ravg;
#print(f"dtheta = {dtheta}")


Ntheta = 121;
dtheta = 2*np.pi/(Ntheta-1);
print(f"Ntheta = {Ntheta}")

## Approaximating the steady state time
t_st = Lc**2/alpha;
print(f"time for steady state approaximately (lumped mass assumtion)  =
{t_st}")

# 3. Initialization and Boundary Conditions
# domain variable
```

```
r = np.linspace(ri, ro, Nr)
theta = np.linspace(0, 2 * np.pi, Ntheta)
time = np.linspace(0, t_st, int(t_st/dt));
## Given temps


T_inner = 120
T_outer_up = 70
T_outer_bottom = 30
T_initial = 30
# domain temps
T = T_initial * np.ones((Nr, Ntheta))
T[0, :] = T_inner;
# Outer radius boundary conditions
for j in range(Ntheta):
    if 0 <= theta[j] <= np.pi:
        T[Nr-1, j] = T_outer_up  # Upper half
    else:
        T[Nr-1, j] = T_outer_bottom # Lower half

# Coordinates for plotting
Theta, R = np.meshgrid(theta, r)
X = R * np.cos(Theta)
Y = R * np.sin(Theta)




#####-------------  Making Non dimentional ----
Phi = (T - T_initial)/(T_inner - T_initial);
R = (r-ri)/(ro-ri);
Theta = theta/(2*np.pi);
Tau = alpha*time/Lc**2;

dR = dr/Lc;
dTheta = dtheta/2*np.pi;
dTau = dt*alpha/Lc**2;
##  tolerance
tol = 1e-7;
Tau_cur = 0.0;
steady = False;
t_plotted = {1: False, 5: False} # Initialize t_plotted here
## loop for the main compuation of the derivatives
while not steady:
  Phi_old = Phi.copy();
  Phi_new = Phi.copy();
 # main logic
  for i in range(1,Nr-1):
      for j in range(Ntheta-1):
```

```
            j_prev = (Ntheta - 2) if j == 0 else (j - 1)
            j_next = j + 1
           # Spatial derivatives
            #d2Phi_dR2 = (Phi[i+1, j] - 2*Phi[i, j] + Phi[i-1, j]) /
dR**2
            #dPhi_dR = (Phi[i+1, j] - Phi[i-1, j]) / (2*dR)
            #d2Phi_dTheta2 = (Phi[i, j_next] - 2*Phi[i, j] + Phi[i,
j_prev]) / dTheta**2

            #
            Cr = dTau/dR**2;
            Ci = dTau/(2*(ri/Lc + R[i])*dR);
            Di = dTau/(((ri/Lc + R[i])**2)*dTheta**2);

            Phi_new[i, j] = (1-2*Cr-2*Di)*Phi[i, j] + (Cr+Ci)*Phi[i+1,
j] + Phi[i-1, j]*(Cr-Ci) + Di*Phi[i, j_next] + (Di)*Phi[i, j_prev] ;

  # as Theta_0 is same as Theta_1 ( 0 == 2*pi)
  Phi_new[:, Ntheta-1] = Phi_new[:, 0];
  Phi = Phi_new;
  Tau_cur = Tau_cur + dTau; #*alpha/Lc**2;
  t_cur = (Tau_cur* Lc**2 )/alpha ;

  # Plot snapshot at t =1s and 5s
  # Plotting check
  for t_mark in [1, 5]:
      if t_cur >= t_mark and not t_plotted[t_mark]:
          plt.figure(figsize=(6, 5))
          cp = plt.contourf(X, Y, Phi, 100, cmap='jet')
          plt.colorbar(cp)
          plt.axis('equal')
          plt.title(f'Non Dimn temp distribution at t = {t_mark}s')
          plt.show()
          t_plotted[t_mark] = True

  # Checking the tolerance condition
  if np.max(np.abs(Phi - Phi_old)) < tol:
      steady = True
      print(f"Steady state reached at t = {t_cur:.2f}s")
      plt.figure(figsize=(6, 5))
      cp = plt.contourf(X, Y, Phi, 100, cmap='jet')
      plt.colorbar(cp)
      plt.axis('equal')
      plt.title(f'Steady State Non Dimn temp distribution(t =
{t_cur:.2f}s)')
      plt.show()
```